

On the Common Substring Alignment Problem

Gad M. Landau, Michal Ziv-Ukelson

Journal of Algorithms 41, pp. 338-359 (2001)

Presenter: Tzu-Chiang Hsu (徐子強)

2015/03/02

Abstract (1/4)

The Common Substring Alignment Problem is defined as follows: Given a set of one or more strings $S_1, S_2 \dots S_c$ and a target string T , Y is a common substring of all strings S_i , that is, $S_i = B_i Y F_i$. The goal is to compute the similarity of all strings S_i with T , without computing the part of Y again and again.

Abstract (2/4)

Using the classical dynamic programming tables, each appearance of Y in a source string would require the computation of all the values in a dynamic programming table of size $O(n\ell)$ where ℓ is the size of Y . Here we describe an algorithm which is composed of an encoding stage and an alignment stage.

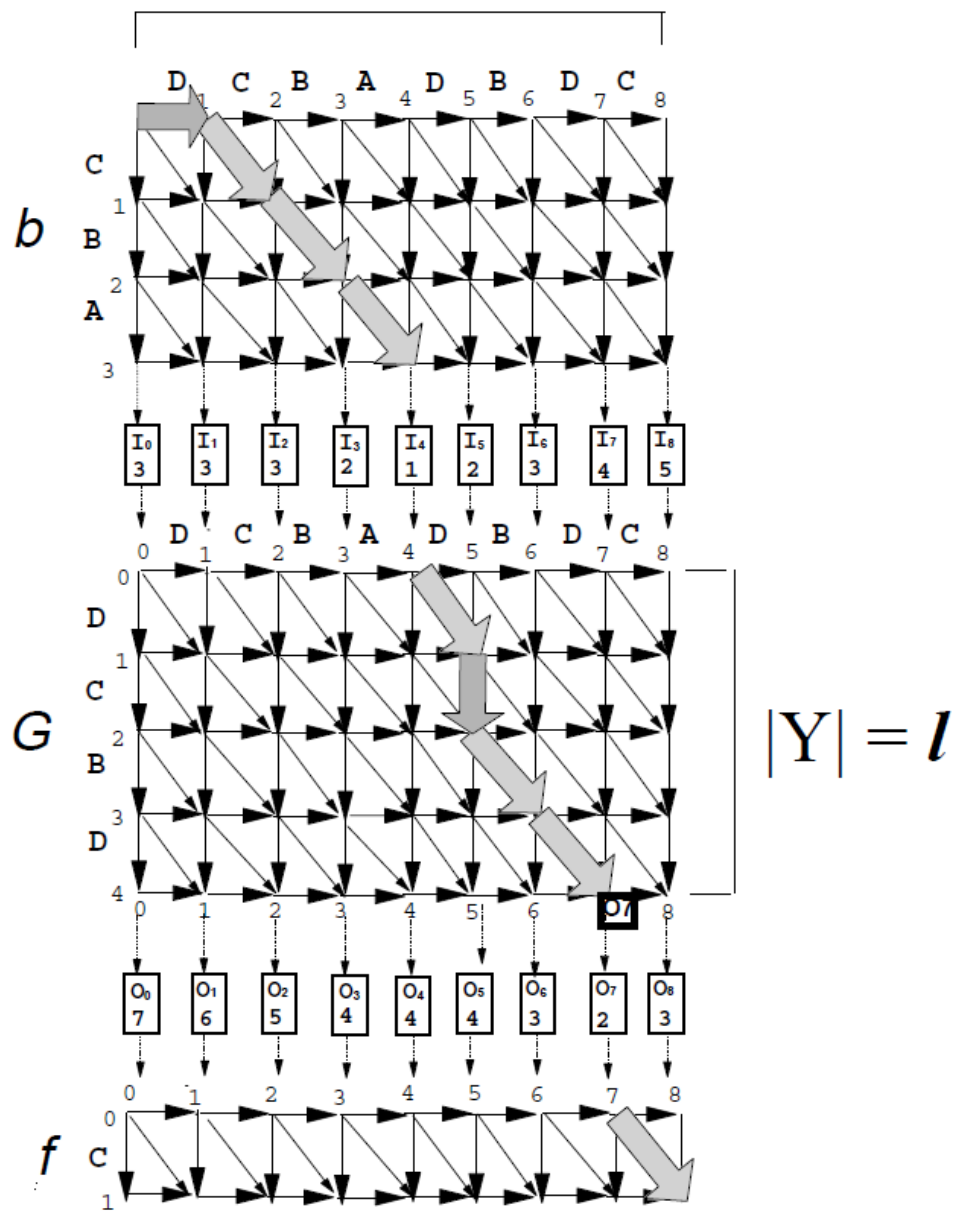
Abstract (3/4)

During the first stage, a data structure is constructed which encodes the comparison of Y with T . Then, during the alignment stage, for each comparison of a source S_i with T , the precompiled data structure is used to speed up the part of Y .

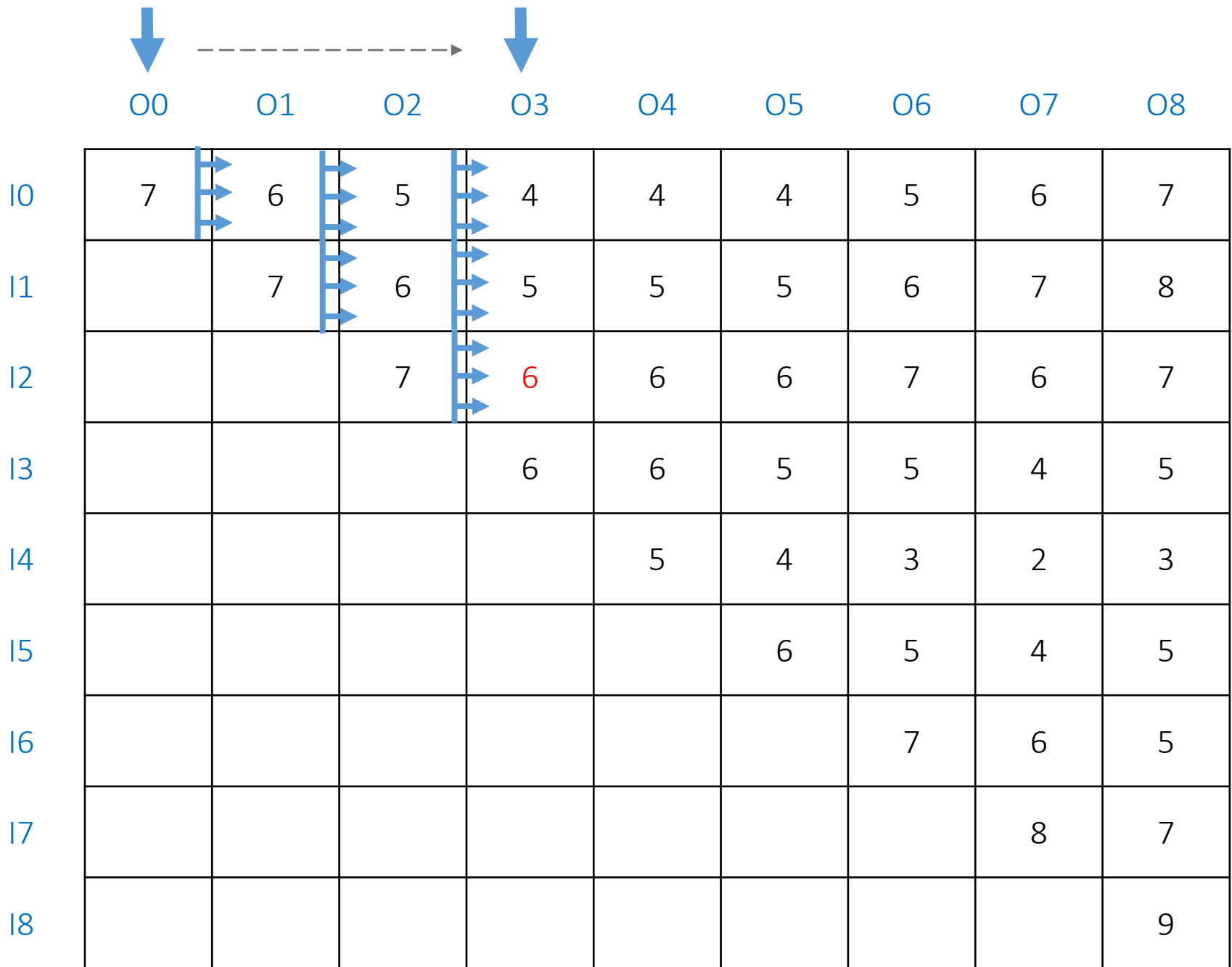
Abstract (4/4)

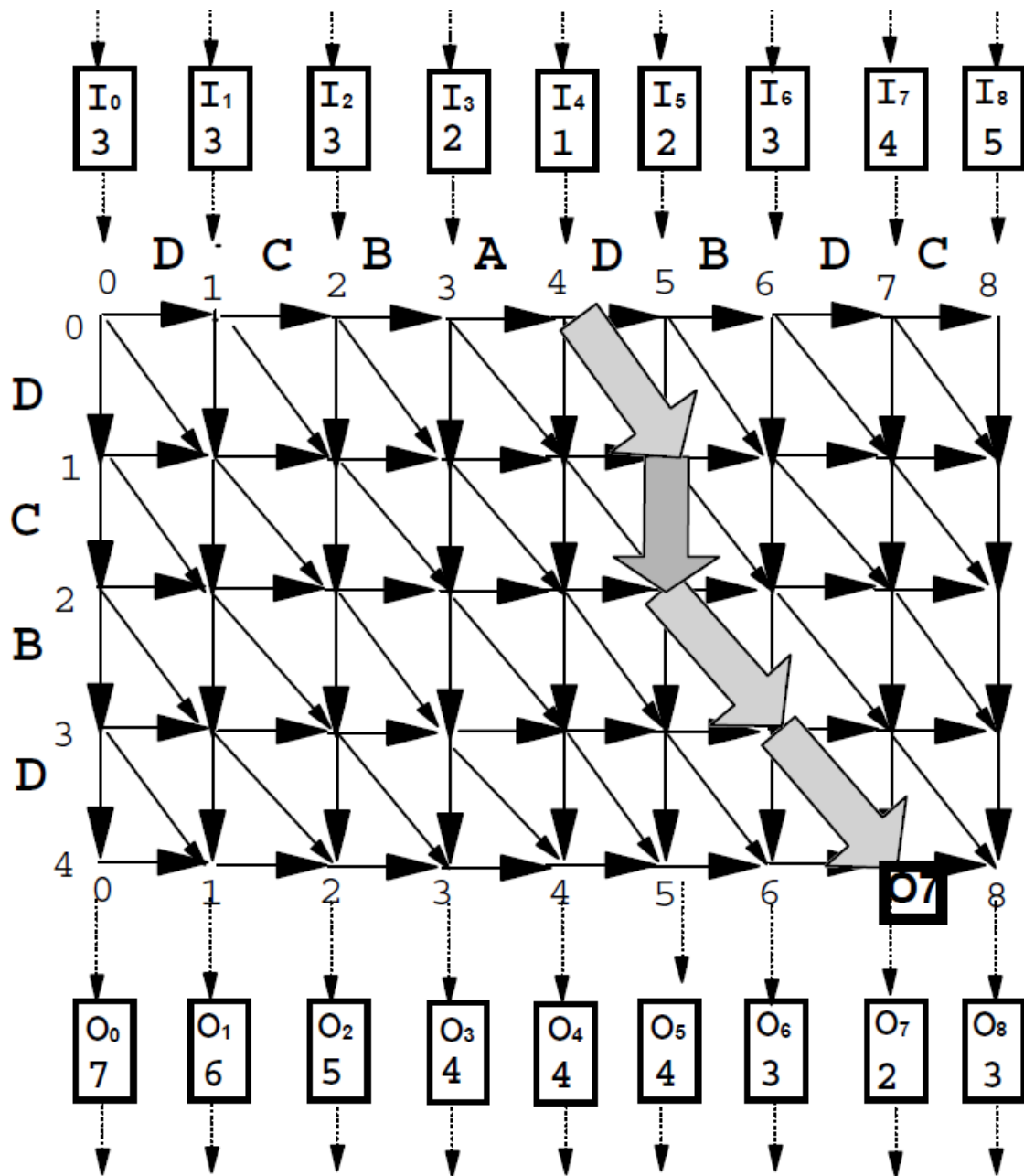
We show how to reduce the $O(n\mathcal{L})$ alignment work, for each appearance of the common substring Y in a source string, to $O(n)$ -at the cost of $O(n\mathcal{L})$ encoding work, which is executed only once.

$$|T| = \mathbf{n}$$



	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	
	3	3	3	2	1	2	3	4	5	<i>input row</i>
	00	01	02	03	04	05	06	07	08	
	<i>DIST</i> matrix									
10	4	3	2	1	1	1	2	3	4	$Edit[T_1^{j=0\dots 9}, Y]$
11	-	4	3	2	2	2	3	4	5	$Edit[T_2^{j=0\dots 9}, Y]$
12	-	-	4	3	3	3	4	3	4	$Edit[T_3^{j=0\dots 9}, Y]$
13	-	-	-	4	4	3	3	2	3	$Edit[T_4^{j=0\dots 9}, Y]$
14	-	-	-	-	4	3	2	1	2	$Edit[T_5^{j=0\dots 9}, Y]$
15	-	-	-	-	-	4	3	2	3	$Edit[T_6^{j=0\dots 9}, Y]$
16	-	-	-	-	-	-	4	3	2	$Edit[T_7^{j=0\dots 9}, Y]$
17	-	-	-	-	-	-	-	4	3	$Edit[T_8^{j=0\dots 9}, Y]$
18	-	-	-	-	-	-	-	-	4	$Edit[T_9^{j=0\dots 9}, Y]$
	0	1	2	3	4	5	6	7	8	<i>column numbers</i>
	<i>OUT</i> matrix:									
10	7	6	5	4	4	4	5	6	7	$OUT[0, j = 0\dots 9]$
11	-	7	6	5	5	5	6	7	8	$OUT[1, j = 0\dots 9]$
12	-	-	7	6	6	6	7	6	7	$OUT[2, j = 0\dots 9]$
13	-	-	-	6	6	5	5	4	5	$OUT[3, j = 0\dots 9]$
14	-	-	-	-	5	4	3	2	3	$OUT[4, j = 0\dots 9]$
15	-	-	-	-	-	6	5	4	5	$OUT[5, j = 0\dots 9]$
16	-	-	-	-	-	-	7	6	5	$OUT[6, j = 0\dots 9]$
17	-	-	-	-	-	-	-	8	7	$OUT[7, j = 0\dots 9]$
18	-	-	-	-	-	-	-	-	9	$OUT[8, j = 0\dots 9]$
	0	1	2	3	4	5	6	7	8	<i>iteration numbers</i>
	O_0	O_1	O_2	O_3	O_4	O_5	O_6	O_7	O_8	
	7	6	5	4	4	4	3	2	3	<i>output row</i>







00

01

02

03

04

05

06

07

08

I0

7



6



5



4

4

4

5

6

7

I1

7



6



5

5

5

6

7

8

I2

7



6

6

6

7

6

7

I3

6

6

5

5

4

5

I4

5

4

3

2

3

I5

6

5

4

5

I6

7

6

5

I7

8

7

I8

9

7	6	5	4	4	4	5	6	7
	7	6	5	5	5	6	7	8
		7	6	6	6	7	6	7
			6	6	5	5	4	5
				5	4	3	2	3
					6	5	4	5
						7	6	5
							8	7
								9



00 01 02 03 04 05 06 07 08

I0

7

6

5

4

4

4

5

6

7

I1

7

6

5

5

5

6

7

8

I2

7

6

6

6

7

6

7

I3

6

6

5

5

4

5

I4

5

4

3

2

3

I5

6

5

4

5

I6

7

6

5

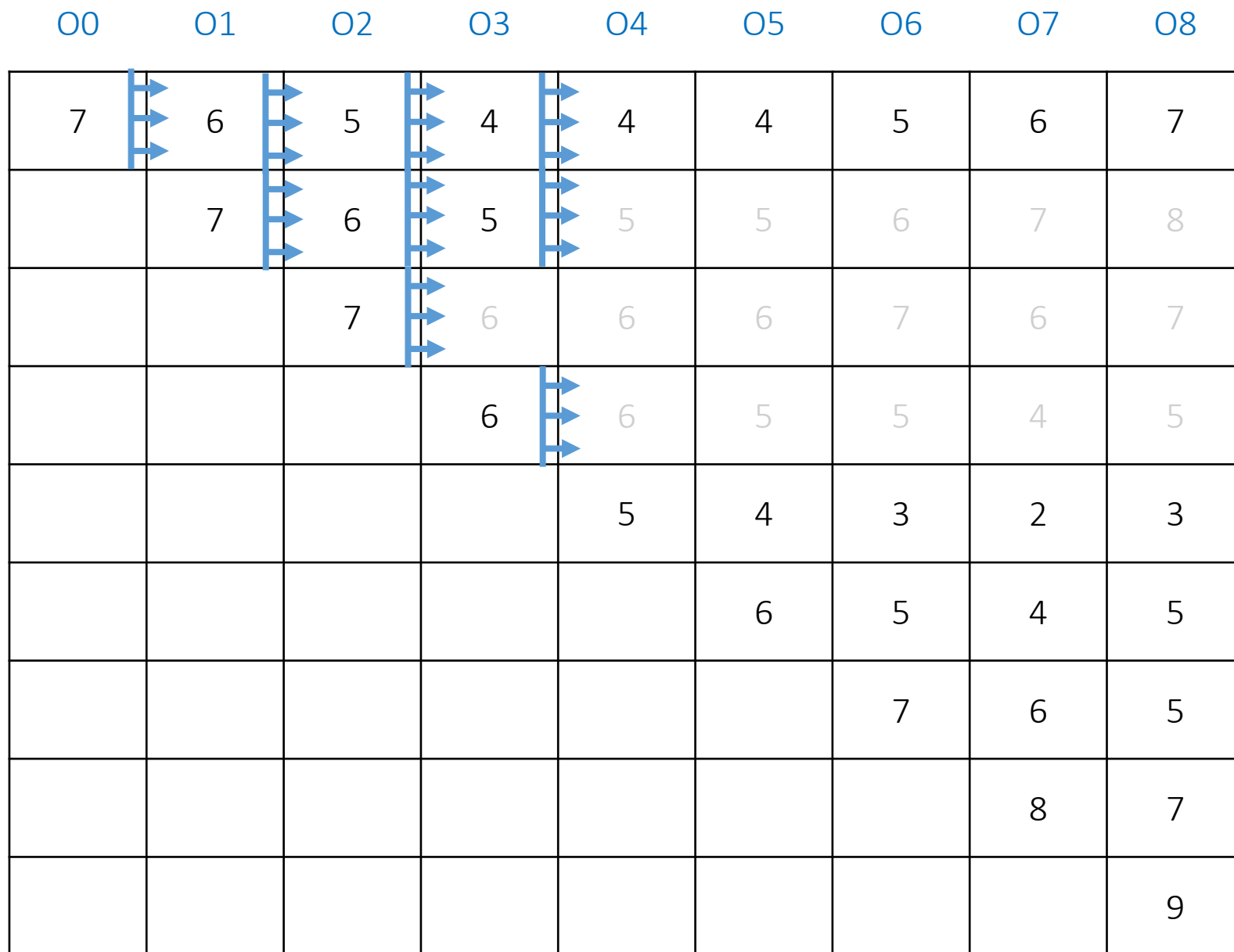
I7

8

7

I8

9





00

01

02

03

04

05

06

07

08

I0

7



6



5



4



4



4

5

6

7

I1

7



6



5



5

5

6

7

8

I2

7



6

6

6

7

6

7

I3

6



6

5

5

4

5

I4

5



4

3

2

3

I5

6

5

4

5

I6

7

6

5

I7

8

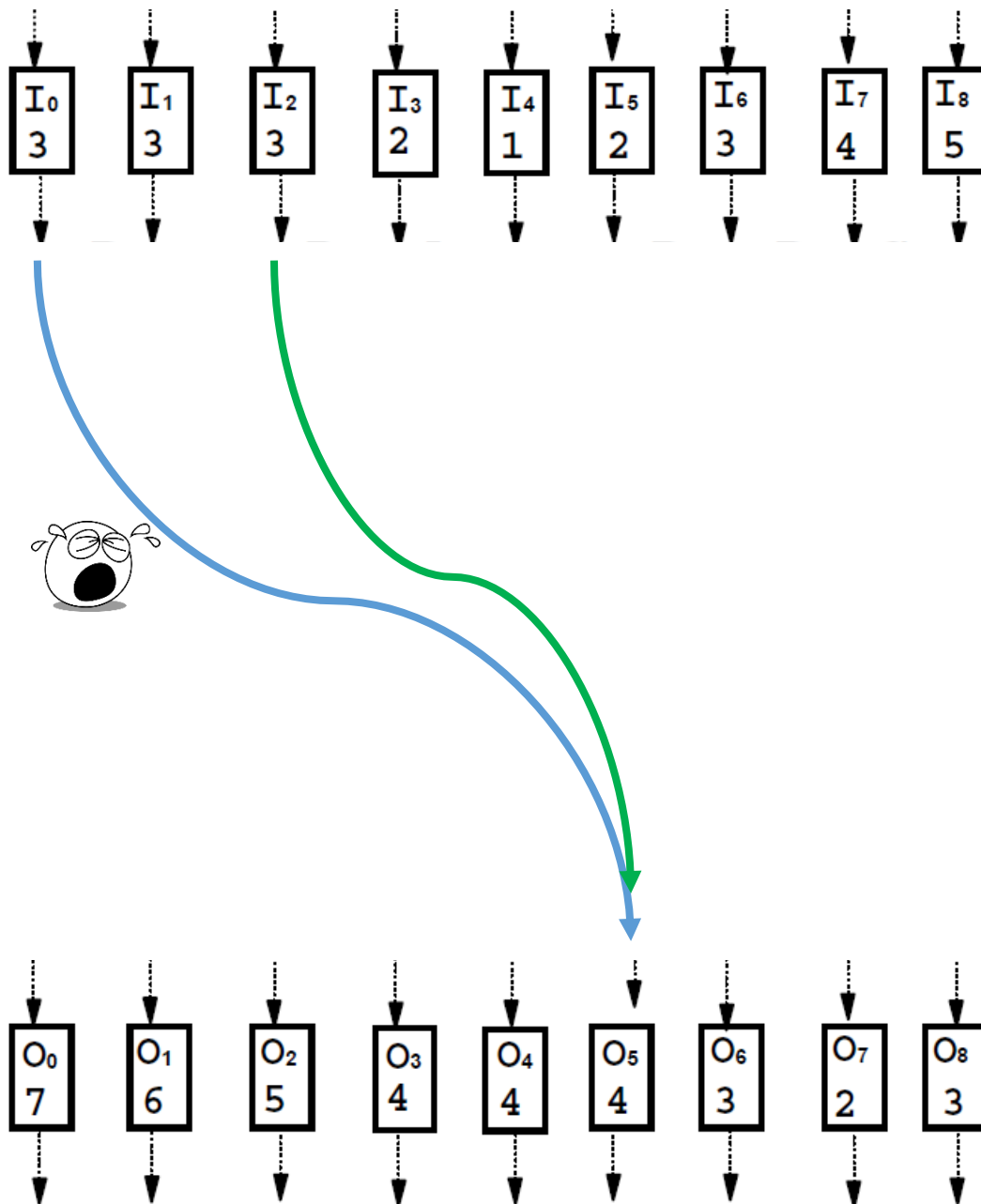
7

I8

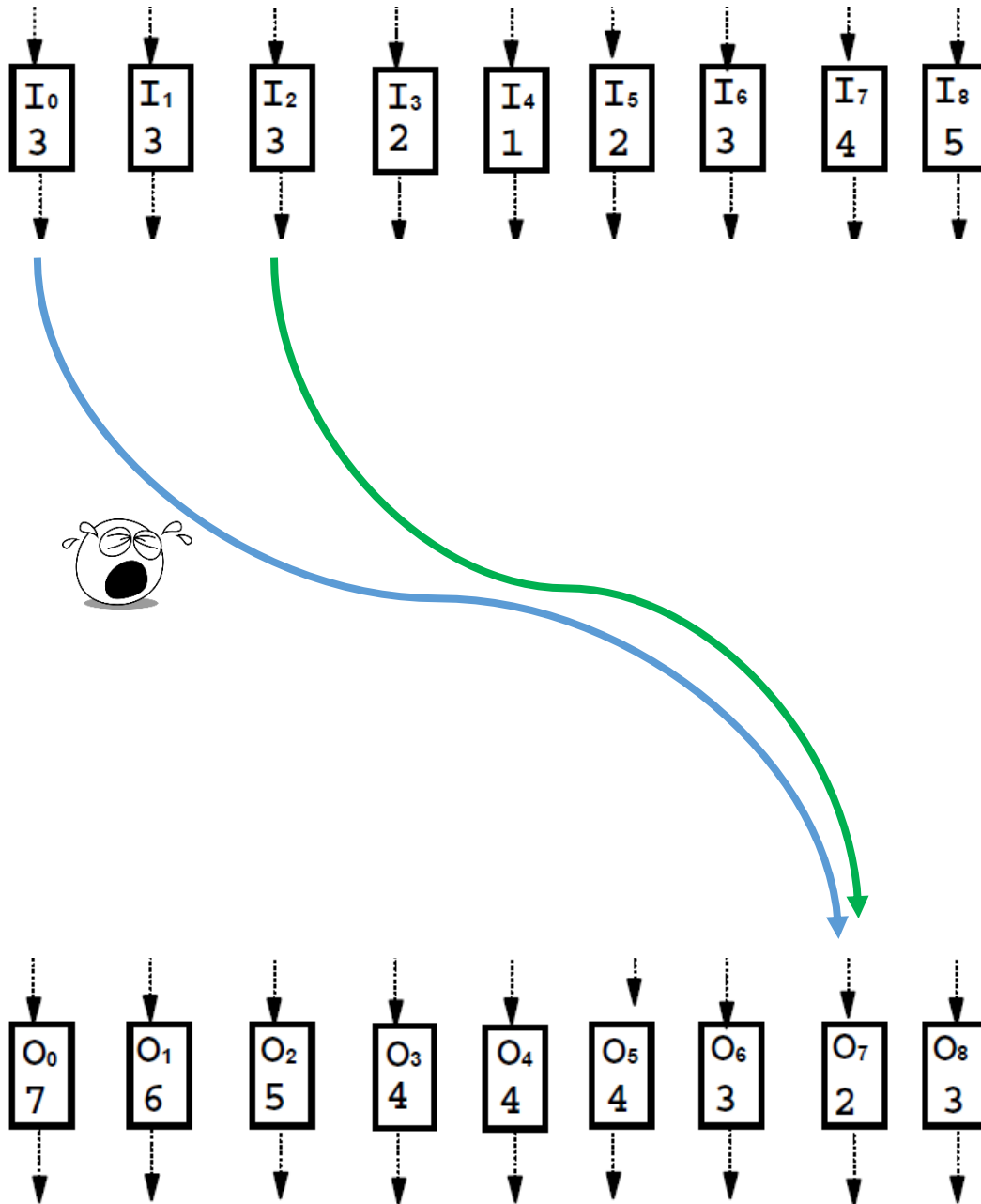
9

[illegible]

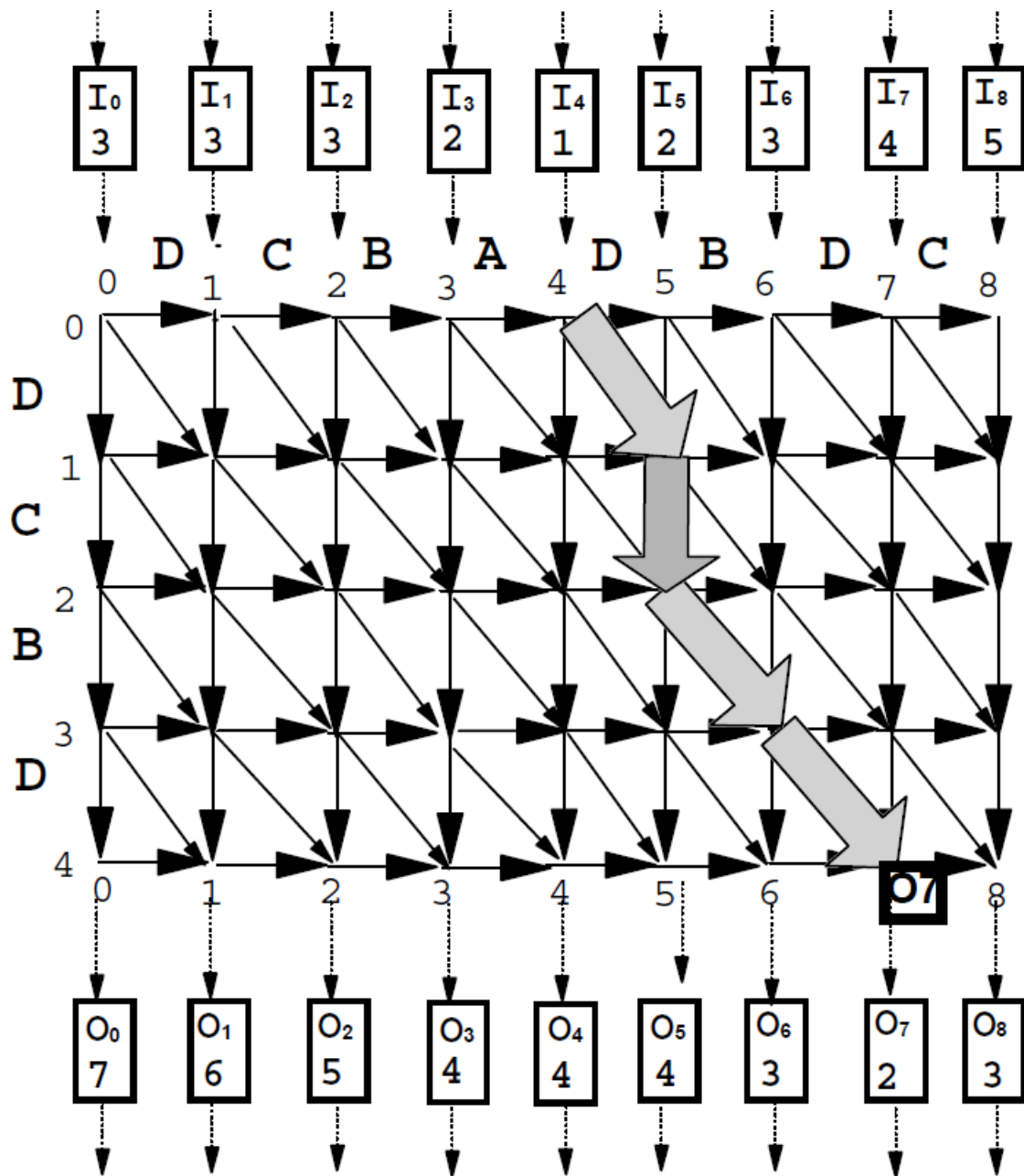
if



then



	00	01	02	03	04	05	06	07	08
10	7	6	5	4	4	4	5	6	7
11		7	6	5	5	5	6	7	8
12			7	6	6	6	7	6	7
13				6	6	5	5	4	5
14					5	4	3	2	3
15						6	5	4	5
16							7	6	5
17								8	7
18									9



	00	01	02	03	04	05	06	07	08
10	7	6	5	4	4	4	5	6	7
11		7	6	5	5	5	6	7	8
12			7	6	6	6	7	6	7
13				6	6	5	5	4	5
14					5	4	3	2	3
15						6	5	4	5
16							7	6	5
17								8	7
18									9

1

1

0

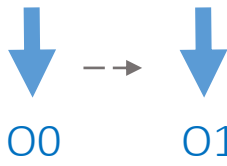
-1

2

2

2

2



	00	01	02	03	04	05	06	07	08
I0	7	6	5	4	4	4	5	6	7
I1		7	6	5	5	5	6	7	8
I2			7	6	6	6	7	6	7
I3				6	6	5	5	4	5
I4					5	4	3	2	3

0	1	2	3	4	5	6	7	8
					2	2	1	5
						3	1	5

	6	5	4	5
		7	6	5
			8	7
				9



O0

O1

O2

O3

O4

O5

O6

O7

O8

I0

7

6

5

4

4

4

5

6

7

1

1

I1

7

6

5

5

5

6

7

8

1

1+1

I2

7

6

6

6

7

6

7

I3

6

6

5

5

4

5

I4

5

4

3

2

3

I5

6

5

4

5

I6

7

6

5

I7

8

7

I8

9



00

01

02

03

04

05

06

07

08

I0

7

6

5

4

4

4

5

6

7

I1

7

1

6

1

5

5

5

6

7

8

I2

1

7

1

6

6

6

7

6

7

I3

0

6

6

5

5

4

5

I4

5

4

3

2

3

I5

6

5

4

5

I6

7

6

5

I7

8

7

I8

9

2+0



00

01

02

03

04

05

06

07

08

I0

7

6

5

4

4

4

5

6

7

I1

7

6

5

5

5

6

7

8

I2

7

1

6

6

6

7

6

7

I3

6

6

5

5

4

5

I4

5

4

3

2

3

I5

6

5

4

5

I6

7

6

5

I7

8

7

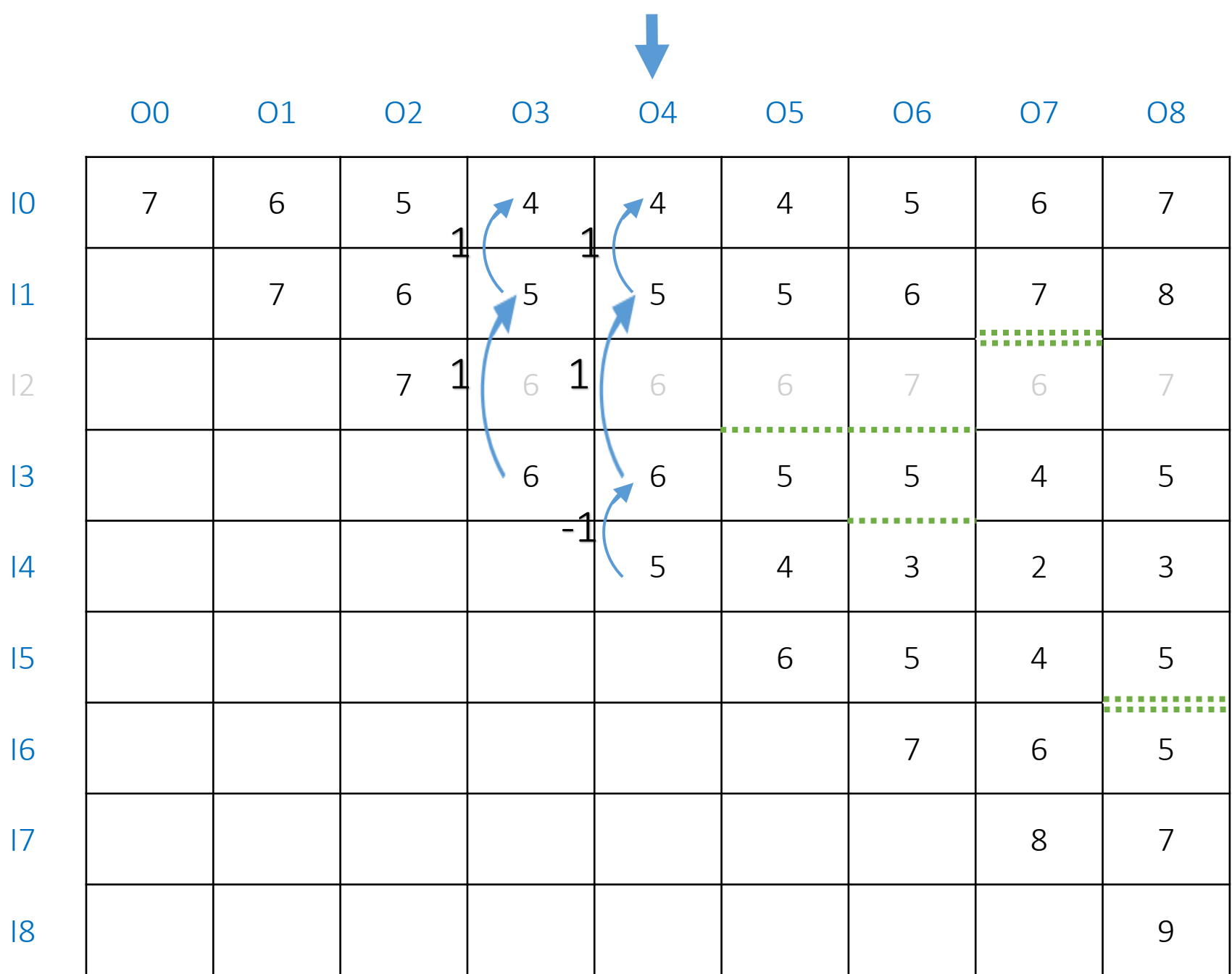
I8

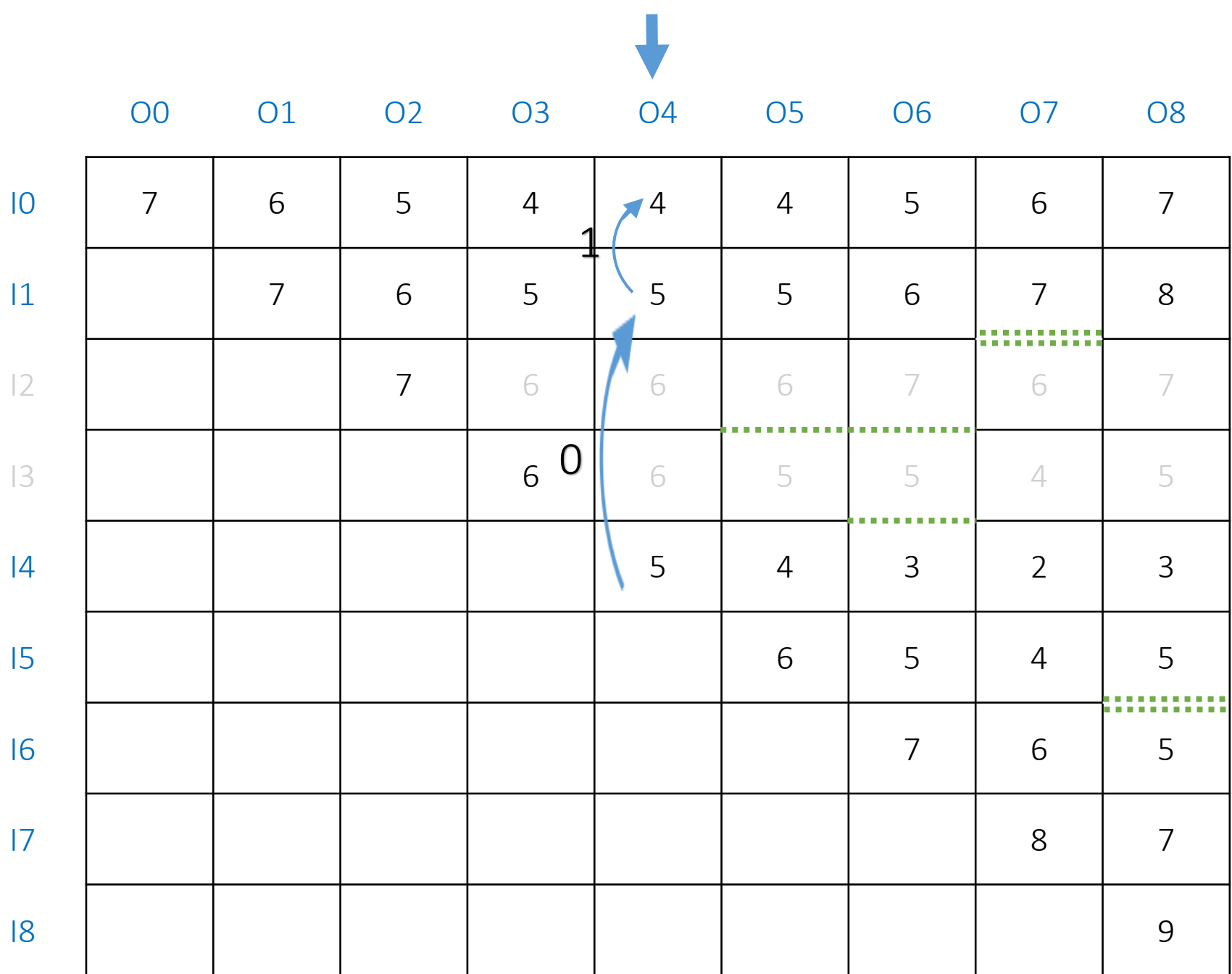
9

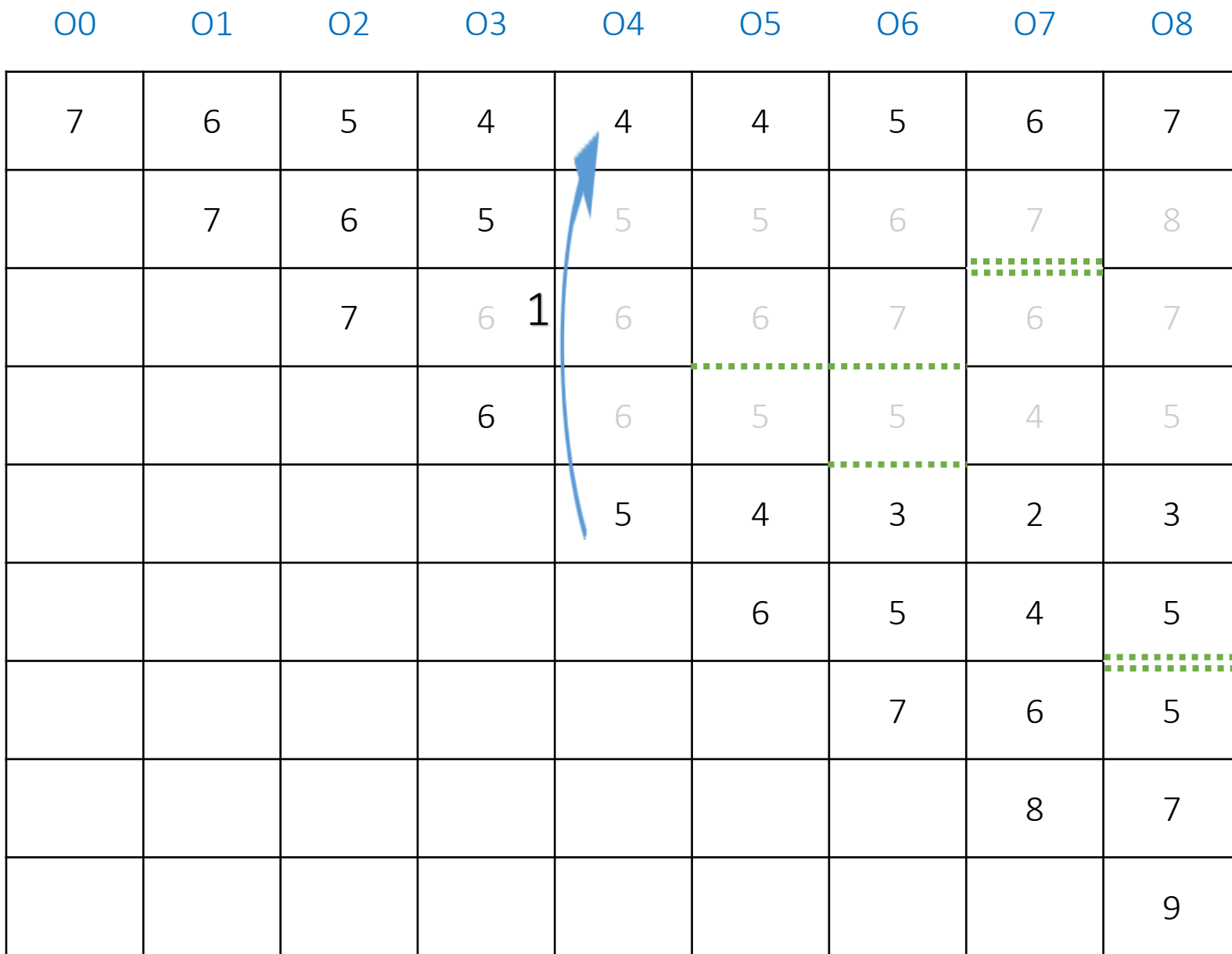
1

1

2







	00	01	02	03	04	05	06	07	08
I0	7	6	5	4	4	4	5	6	7
I1		7	6	5	5	5	6	7	8
I2			7	6	5	6	7	6	7
I3				6	6	5	5	4	5
I4					5	4	3	2	3
I5						6	5	4	5
I6							7	6	5
I7								8	7
I8									9



1-1=0

2





00

01

02

03

04

05

06

07

08

I0

7

6

5

4

4

4

5

6

7

I1

7

6

5

5

5

6

7

8

I2

7

6

6

6

7

6

7

I3

6

6

5

5

4

5

I4

5

4

3

2

3

I5

2

6

5

4

5

I6

7

6

5

I7

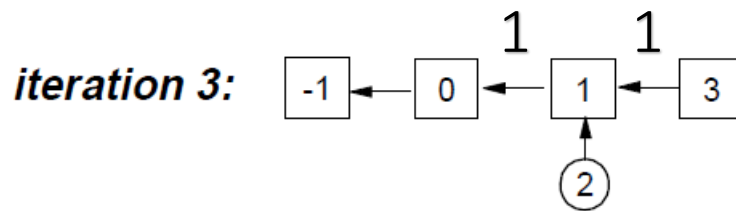
8

7

I8

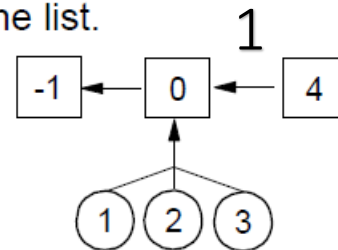
9





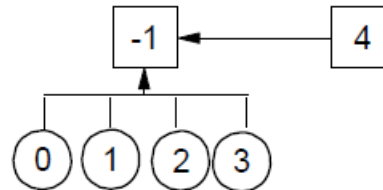
iteration 4:

Event 1: Row 4 joins the list.

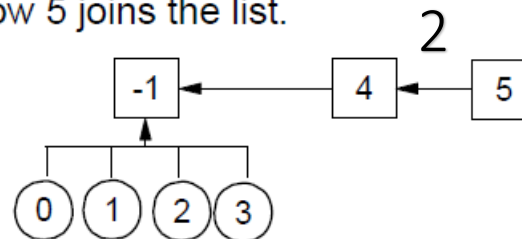


iteration 5:

Event 2: Borderline[5,1]=2.



Event 1: Row 5 joins the list.



Borderline Points:

0	1	2	3	4	5	6	7	8
					2	2	1	5
						3	1	5

$O(\Psi n + n + n)$ time